Universidad Autónoma de Tamaulipas
Unidad Académica Multidisciplinaria Mante Centro

Nombre del trabajo:
Reporte práctica 5

Asignatura:
Arquitectura de Equipos Digitales

Profesor:
López piña Daniel

Alumno:
Guerrero Mendoza Emmanuel

Grado: 6° **Grupo:** EF

Ciudad Mante, Tamaulipas Abril 2026

Introducción

La comunicación serial UART (Universal Asynchronous Receiver Transmitter) es un método fundamental para el intercambio de datos entre dispositivos electrónicos. En sistemas embebidos, esta comunicación permite la interacción directa entre un microcontrolador y una computadora, facilitando el envío de comandos y la recepción de respuestas en tiempo real.

En esta práctica se implementó un sistema de comunicación serial bidireccional utilizando Arduino, donde el usuario puede enviar comandos desde el monitor serial y el microcontrolador los interpreta para ejecutar diferentes acciones. Además, se incorporó un parser de comandos que permite procesar instrucciones de forma estructurada, simulando el funcionamiento de sistemas más complejos.

Desarrollo

Para el desarrollo de la práctica se utilizó un Arduino basado en el microcontrolador ATmega328P, configurando la comunicación serial con la siguiente instrucción:

- Serial.begin(115200);

Se eligió una velocidad de transmisión de 115200 baudios, lo que permite una comunicación más rápida en comparación con valores estándar como 9600.

Inicialización del sistema.

Durante la función setup(), se configuró el pin 13 como salida para controlar un LED y se enviaron mensajes de bienvenida al usuario mediante el monitor serial. Estos mensajes incluyen información del sistema, como el baud rate, formato de comunicación (8N1) y los comandos disponibles:

- LED_ON
- LED_OFF
- STATUS
- BLINK:N

Esto permite que el usuario conozca cómo interactuar con el sistema desde el inicio.

Recepción de datos

En la función loop(), se implementó la lectura de datos provenientes del puerto serial mediante un ciclo:

- While (Serial.available() > 0)

Cada carácter recibido es leído con Serial.read() y almacenado en una variable tipo String llamada bufferCmd. Esto permite construir comandos completos carácter por carácter.

Se estableció que el final de un comando ocurre cuando se detecta un salto de línea ('\n'). En ese momento, el comando acumulado es enviado a la función procesarComando() para su análisis.

Procesamiento de comandos (Parser)

La función procesarComando(String cmd) es el núcleo del sistema, ya que interpreta los comandos enviados por el usuario.
Primero, se limpia el comando utilizando:

- Cmd.trim();

Esto elimina espacios innecesarios y evita errores en la comparación.
Posteriormente, se evalúan diferentes condiciones:

Comando LED_ON

Enciende el LED conectado al pin 13 y actualiza la variable de estado:

- digitalWrite(LED_PIN, HIGH);

Comando LED_OFF

Apaga el LED y actualiza su estado:

- digitalWrite(LED_PIN, LOW);

Comando STATUS

Muestra información del sistema:

- Estado del LED (ON/OFF)
- Tiempo de ejecución (millis())
- Memoria RAM disponible

Esto permite monitorear el comportamiento del sistema en tiempo real.

Comando BLINK:N

Este comando permite hacer parpadear el LED un número específico de veces. El valor N se obtiene mediante:

- Cmd.substring(6).toInt();

Se ejecuta un ciclo for que enciende y apaga el LED con pequeños retardos (delay(150)), generando el efecto de parpadeo.

Manejo de errores

Si el comando no coincide con ninguno de los anteriores, el sistema responde con un mensaje de error indicando que el comando es desconocido.

Función de memoria (freeRam)

Se implementó una función adicional llamada freeRam() que calcula la memoria SRAM disponible en el sistema. Esto se logra mediante direcciones de memoria internas del microcontrolador, lo cual es una técnica avanzada para monitoreo de recursos.

BlinkBasico | Arduino IDE 2.3.8

File Edit Sketch Tools Help

Arduino UNO

BlinkBasico.ino

```
//·lab05_uart_comandos.ino
//·Comunicacion·UART·bidireccional·con·parser·de·comandos
const·int·LED_PIN·=·13;
bool·ledEstado·=·false;
String·bufferCmd·=·"";
void·setup()·{
·Serial.begin(115200);
·pinMode(LED_PIN,·OUTPUT);
·//·Mensaje·de·bienvenida·con·datos·del·sistema
·Serial.println("============================");
·Serial.println("Sistema·UART·-·Arduino/ATmega");
·Serial.println("Baud·rate:·115200·|·Formato:·8N1");
```

Output Serial Monitor X

Message (Enter to send message to 'Arduino UNO' on 'COM4') Both NL & CR 115200 baud

```
============================
Sistema UART - Arduino/ATmega
Baud rate: 115200 | Formato: 8N1
Comandos: LED_ON, LED_OFF, STATUS, BLINK:N
============================
[CMD RECIBIDO]: 'LED_ON'
[OK] LED encendido
[CMD RECIBIDO]: 'LED_OFF'
[OK] LED apagado
[CMD RECIBIDO]: 'STATUS'
--- STATUS ---
LED: OFF
Uptime: 19 s
Free SRAM aprox: 1486 bytes
[CMD RECIBIDO]: 'BLINK:3'
[OK] Blink x3
[CMD RECIBIDO]: 'HOLA'
[ERROR] Comando desconocido: 'HOLA'
```

Ln 68, Col 1 Arduino UNO on COM4

Resultados

El sistema desarrollado permitió establecer una comunicación serial bidireccional eficiente entre el Arduino y la computadora. Se comprobó que:

Los comandos son correctamente recibidos y procesados
El LED responde a las instrucciones enviadas
El sistema puede mostrar información en tiempo real
Se pueden ejecutar comandos dinámicos como BLINK:N
La memoria y el tiempo de ejecución pueden ser monitoreados

Esto demuestra que el sistema no solo responde a comandos simples, sino que también puede manejar funciones más complejas.

PREGUNTAS DE REFLEXION

¿Cuántos bits se transmiten por carácter en formato 8N1 a 115200 bps? ¿Cuánto
Tiempo tarda en transmitirse la cadena 'LED_ON' completa?
R=

- Bits por carácter (8N1): 10 bits
- Cadena "LED_ON": 60 bits
- Tiempo: ≈ 0.52 ms

¿Qué ocurre si el Arduino está configurado a 115200 bps pero el monitor serial está a 9600 bps? Prueba este escenario y describe el comportamiento.
R=Se reciben datos corruptos o ilegibles aparecen símbolos extraños (basura) la comunicación no funciona correctamente

¿Por qué se usa un buffer de String para acumular los caracteres del comando en lugar de procesar cada byte individualmente al recibirlo?
R= Permite formar comandos completos facilita la comparación de strings hace el código más simple y organizado

El comando 'BLINK:N' usa delay() dentro del procesador de comandos. ¿Qué problema tiene esto? ¿Cómo rediseñarías el sistema para que el Arduino pueda recibir otros comandos mientras parpadea?
R=Problema: bloquea el Arduino (no ejecuta otras tareas)
Solución: usar millis() (programación no bloqueante)

Como extenderías el protocolo para que el Arduino envíe datos de sensor cada 2 segundos Y al mismo tiempo procese comandos entrantes? Describe la arquitectura del firmware.
R=

- Usar millis() para temporización

Separar tareas:

- Recepción UART
- Envío periódico cada 2 segundos

- Ejecutar ambas en el loop() sin delay()

Conclusión

La práctica permitió comprender a fondo el funcionamiento de la comunicación UART y su aplicación en sistemas embebidos. Se desarrolló un sistema interactivo capaz de interpretar comandos mediante un parser, lo cual representa un nivel más avanzado en comparación con ejemplos básicos.

Además, se integraron conceptos importantes como manejo de memoria, procesamiento de cadenas y control de hardware, demostrando la versatilidad del Arduino para el desarrollo de soluciones prácticas.

Bibliografía

- Arduino. (s.f.). Arduino Documentation. Recuperado de https://www.arduino.cc
- Microchip Technology Inc. (s.f.). ATmega328P Datasheet.
- Docente de la asignatura. (2026). Arquitectura de Equipos Digitales: Prácticas de laboratorio 1–12.